

# MODELE OBLICZEŃ

NOTATKI

---

*„Liczba nieparzysta to taka, że zawsze jest jeden, chyba że jest zero.”*

## Spis treści

|                                |    |
|--------------------------------|----|
| 1. Języki                      | 2  |
| 2. Automaty skończone          | 4  |
| 3. Wyrażenia regularne         | 7  |
| 4. Języki regularne            | 8  |
| 5. Problemy decyzyjne          | 12 |
| 6. Gramatyki bezkontekstowe    | 13 |
| 7. Postać normalna Chomsky'ego | 14 |
| 8. Automaty ze stosem          | 16 |
| 9. Języki bezkontekstowe       | 18 |

# 1. Języki

**Notacja.** Będziemy rozważać alfabet  $\Sigma$  i słowa zbudowane z jego elementów. Zazwyczaj interesuje nas  $\Sigma = \{0, 1\}$ . Wprowadzamy oznaczenia:

- $\Sigma^i$  – zbiór słów długości  $i$ .
- $\Sigma^* = \bigcup_{i=0}^{\infty} \Sigma^i$  – zbiór wszystkich słów nad  $\Sigma$ .
- $\Sigma^+ = \bigcup_{i=1}^{\infty} \Sigma^i$  – zbiór niepustych słów.
- $\varepsilon$  – puste słowo (czyli  $\Sigma^0 = \{\varepsilon\}$ ).

**Definicja 1.** Językiem nad  $\Sigma$  nazywamy dowolne  $L \subseteq \Sigma^*$  – podzbiór słów. Będzie nas interesować, które nieskończone języki da się opisać w skończony sposób (to znaczy stworzyć program, który dla danego słowa stwierdza, czy należy ono do języka).

**Uwaga.** Programy są skończone, więc jest ich przeliczalnie wiele. Języków jest natomiast nieprzeliczalnie wiele. Zatem muszą istnieć języki, których nie da się opisać.

**Przykład.** Mamy problem decyzyjny: czy liczba  $x$  jest parzysta. Zapisując liczbę w postaci binarnej widzimy, że liczby naturalne są podzbiorem  $\Sigma^*$  (językiem), a nasze pytanie sprowadza się do sprawdzenia, czy dane słowo należy do tego języka. Podobnie możemy mówić np. o problemach grafowych kodując graf jako macierz incydencji.

**Notacja.** Słowa z  $\Sigma^n$  są właściwie (na co wskazuje notacja) funkcjami  $n \rightarrow \Sigma$ . Będziemy zatem często pisać  $v(i)$  dla  $v \in \Sigma^n$  i  $i \in n$ . Wprowadzamy też oznaczenie na rozmiar słowa  $|v|$ .

**Definicja 2.** Konkatenacją słów nazywamy funkcję  $\cdot : \Sigma^* \times \Sigma^* \rightarrow \Sigma^*$  zadaną przez

$$(v \cdot w)(i) = \begin{cases} v(i), & i < |v| \\ w(i - |v|), & |v| \leq i < |v| + |w| \end{cases}.$$

Jest to po prostu sklejenie dwóch słów. W zapisie często pomijamy kropkę i piszemy  $vw$ .

**Definicja 3.** Słowo  $x$  jest prefiksem słowa  $v$ , jeśli istnieje takie  $y \in \Sigma^*$ , że  $xy = v$ . Oznaczamy to  $x \sqsubseteq v$ . Prefiks jest właściwy, jeśli nie jest pełnym słowem – to oznaczamy  $x \sqsubset v$ . Analogicznie definiujemy pojęcie sufiksu.

**Propozycja 1.** Konkatenacja spełnia własności:

- łączność  $(ab)c = a(bc)$ .
- $\varepsilon$  jest elementem neutralnym  $\varepsilon a = a\varepsilon = a$ .
- $|ab| = |a| + |b|$ .

Zatem  $(\Sigma^*, \cdot, \varepsilon)$  jest monoidem.

**Definicja 4.** Definiujemy potęgę słowa  $w$  jako

$$w^n = \begin{cases} \varepsilon, & n = 0 \\ w^{n-1}w, & n > 0 \end{cases}.$$

**Notacja.** Na językach  $L_1, L_2$  możemy wprowadzić operacje teoriomnogościowe sumy  $L_1 \cup L_2$ , przecięcia  $L_1 \cap L_2$  i dopełnienia  $\Sigma^* \setminus L_1$ .

**Definicja 5.** Katenacją języków  $L_1, L_2$  nazywamy zbiór (język)  $L_1 \cdot L_2 = \{w_1 w_2 : w_1 \in L_1, w_2 \in L_2\}$ . Mamy  $L\emptyset = \emptyset L = \emptyset$  i  $L\{\varepsilon\} = \{\varepsilon\}L = L$ .

**Definicja 6.** Definiujemy potęgę języka  $L$  jako

$$L^n = \begin{cases} \{\varepsilon\}, & n = 0 \\ L^{n-1}L, & n > 0 \end{cases}.$$

**Definicja 7.** Gwiazdką Kleene'ego nazywamy operację

$$L^* = \bigcup_{i=0}^{\infty} L^i,$$

gdzie  $L$  jest językiem. Definiujemy też operację

$$L^+ = \bigcup_{i=1}^{\infty} L^i.$$

**Lemat 1 (Lemat Ardena).** Niech  $A, B, X \subseteq \Sigma^*$  będą związane równaniem  $AX \cup B = X$  ( $XA \cup B = X$ ). W takiej sytuacji język  $X = A^*B$  ( $X = BA^*$ ) jest najmniejszym (w sensie inkluzji) rozwiązaniem tego równania. Jeśli  $\varepsilon \notin A$ , to jest to jedyne rozwiązanie.

**Dowód.**

$$A(A^*B) \cup B = A((A^+ \cup \{\varepsilon\})B) \cup B = A(A^+B \cup B) \cup B = AA^+B \cup AB \cup B = A^*B.$$

Niech  $X$  będzie rozwiązaniem. Mamy  $A^*B = \bigcup_{i=0}^{\infty} A^iB$ . Indukcją po  $i$  pokażemy, że  $A^iB \subseteq X$ . Mamy  $A^0B = B \subseteq X$ . Jeśli  $A^iB \subseteq X$ , to z naszego równania mamy  $AA^iB = A^{i+1}B \subseteq X$ . Z tego wynika minimalność naszego rozwiązania.

Założmy, że  $\varepsilon \notin A$  i istnieje  $v \in X \setminus A^*B$  będące najkrótszym słowem w tym zbiorze. W szczególności  $v \notin B$ , a więc  $v = uw$ , gdzie  $u \in A$ ,  $w \in X$ . Mamy  $w \notin A^*B$ , bo inaczej  $v \in AA^*B \subseteq A^*B$ . Zatem  $w$  spełnia te same założenia co  $v$  i jest ostro krótsze, bo  $\varepsilon \notin A$ . Uzyskana sprzeczność kończy dowód.

Drugą wersję lematu dowodzimy analogicznie. □

**Notacja.** Wprowadzamy oznaczenie  $a \leq b$  oznaczające  $a + b = b$ .

**Definicja 8.** Algebrą Kleene'ego nazywamy krotkę  $(A, +, \cdot, *, 0, 1)$  spełniającą własności:

1.  $(a + b) + c = a + (b + c)$ .
2.  $a + b = b + a$ .
3.  $a + a = a$ .
4.  $a + 0 = a$ .
5.  $(ab)c = a(bc)$ .
6.  $a \cdot 1 = 1 \cdot a = a$ .
7.  $a \cdot 0 = 0 \cdot a = 0$ .
8.  $(b + c)a = ba + ca$ .
9.  $a(b + c) = ab + ac$ .
10.  $1 + aa^* \leq a^*$ .
11.  $1 + a^*a \leq a^*$ .
12.  $ax + b \leq x \implies a^*b \leq x$ .
13.  $xa + b \leq x \implies ba^* \leq x$ .

**Propozycja 2.** Krotka  $(\mathcal{P}(\Sigma^*), \cup, \cdot, *, \emptyset, \{\varepsilon\})$  jest algebrą Kleene'ego.

**Twierdzenie 1 (Kozen).** Każde równanie prawdziwe w algebrze Kleene'ego jest dowodliwe z jej aksjomatów.

**Propozycja 3.**  $\leq$  w algebrze Kleene'ego jest porządkiem częściowym. Ponadto wszystkie operacje są względem niego monotoniczne: dla dowolnych  $a, b, c$  będących elementami algebry Kleene'ego zachodzi

$$\begin{aligned}a \leq b &\implies ac \leq bc, \\a \leq b &\implies ca \leq cb, \\a \leq b &\implies a + c \leq b + c, \\a \leq b &\implies a^* \leq b^*.\end{aligned}$$

**Dowód.** Zwrotność daje nam aksjomat  $a + a = a$ . Zakładając  $x \leq y \leq z$  mamy

$$x + z = x + (y + z) = (x + y) + z = y + z = z,$$

a więc  $x \leq z$  – przechodność. Zakładając  $x \leq y, y \leq x$  mamy

$$y = x + y = y + x = x,$$

a to jest antysymetryczność. Teraz rozpisujemy

$$\begin{aligned}ac + bc &= (a + b)c = bc, \\ca + cb &= c(a + b) = cb \\a + c + b + c &= (a + b) + (c + c) = b + c,\end{aligned}$$

To są trzy pierwsze wynikania. Z nich mamy pierwszą nierówność w  $1 + ab^* \leq 1 + bb^* \leq b^*$ , a to daje nam  $a^* = a^* \cdot 1 \leq b^*$ .  $\square$

**Lemat 2.** W algebrze Kleene'ego mamy

$$ax \leq xb \implies a^*x \leq xb^*.$$

**Dowód.** Z założenia dostajemy  $axb^* \leq xbb^*$ , a więc

$$x + a(xb^*) \leq x + xbb^* = x(1 + bb^*) \leq xb^*.$$

Nierówność  $x + a(xb^*) \leq xb^*$  daje nam  $a^*x \leq xb^*$ .  $\square$

## 2. Automaty skończone

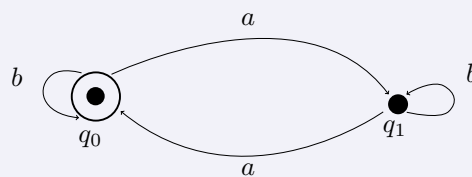
2025-02-20

**Definicja 9.** Deterministyczny automat skończony (DFA – deterministic finite automaton) to krotka  $A = (Q, \Sigma, \delta, s, F)$ , gdzie:

- $Q$  – skończony zbiór stanów,
- $\Sigma$  – skończony alfabet,
- $\delta : Q \times \Sigma \rightarrow Q$  – funkcja przejścia,
- $s \in Q$  – stan początkowy,
- $F \subseteq Q$  – stany końcowe (akceptujące).

Automat zaczyna od zadanego stanu  $s$  i dostaje kolejne litery alfabetu, na podstawie których przechodzi do nowych stanów w sposób zadany przez  $\delta$ . Będą nas interesować sytuacje, w których ostatecznie znajdziemy się w stanie należącym do  $F$ .

**Przykład.** DFA łatwo przedstawia się za pomocą grafu, na przykład takiego jak na Rysunku 1.



Rysunek 1: Graf DFA o stanach  $\{q_0, q_1\}$  i alfabecie  $\{a, b\}$ . Mamy stan początkowy  $q_0$  i zbiór stanów końcowych  $\{q_0\}$ . Jest też  $\delta = \{(q_0, b, q_0), (q_1, b, q_1), (q_0, a, q_1), (q_1, a, q_0)\}$ .

**Pomysł.** Dla zadanego DFA będziemy rozważać słowa, które po przepuszczeniu przez DFA produkują stan końcowy. Będziemy zajmować się językami opisanymi w taki sposób.

**Definicja 10.** Definiujemy funkcję  $\hat{\delta} : Q \times \Sigma^* \rightarrow Q$  w następujący sposób

$$\hat{\delta}(q, w) = \begin{cases} q, & w = \varepsilon \\ \delta(\hat{\delta}(q, x), a), & w = xa, a \in \Sigma, x \in \Sigma^* \end{cases}$$

Jest to funkcja określająca stan automatu po wielu krokach (zadanych danym słowem).

**Definicja 11.** Niech  $A = (Q, \Sigma, \delta, s, F)$  będzie DFA. Językiem akceptowanym przez  $A$  nazywamy język  $L(A) = \{w \in \Sigma^* : \hat{\delta}(s, w) \in F\}$ .

**Przykład.** Dla DFA z Rysunku 1 zachodzi  $L(A) = \{w \in \{a+b\}^* : \#_a(w) = 0 \pmod{2}\}$ .

**Dowód.** Oznaczmy  $L' = \{w \in \{a+b\}^* : \#_a(w) = 0 \pmod{2}\}$ . Indukcją po liczbie  $a$  w słowie  $w \in L'$  dowodzimy, że  $\hat{\delta}(q_0, w) = q_0$  oraz  $\hat{\delta}(q_1, w) = q_1$ . Z pierwszej równości wynika  $L' \subseteq L(A)$ . Z drugiej natomiast wynika, że  $L'^c \cap L(A) = \emptyset$ . To daje nam tezę.

**Definicja 12.** Niedeterministyczny automat skończony (NFA – nondeterministic finite automaton) to krotka  $A = (Q, \Sigma, \delta, S, F)$ , gdzie:

- $Q$  – skończony zbiór stanów,
- $\Sigma$  – skończony alfabet,
- $\delta \subseteq Q \times \Sigma \times Q$  – relacja przejścia,
- $S \subseteq Q$  – zbiór stanów początkowych,
- $F \subseteq Q$  – stany końcowe (akceptujące).

Automat niedeterministyczny różni się od deterministycznego tym, że zaczyna w wielu stanach początkowych i wykonuje przejścia zgodne z  $\delta$ , która tym razem jest relacją – dla zadanego stanu i litery może być wiele poprawnych przejść (lub nie być żadnego).

**Definicja 13.** Definiujemy funkcję  $\tilde{\delta} : \mathcal{P}(Q) \times \Sigma \rightarrow \mathcal{P}(Q)$  w następujący sposób

$$\tilde{\delta}(B, a) = \{q \in Q : \exists q_b \in B \ (q_b, a, q) \in \delta\}.$$

Jest to funkcja określająca możliwe stany dla zadaných stanów początkowych i litery.

**Definicja 14.** Analogicznie jak przedtem definiujemy funkcję  $\hat{\delta} : \mathcal{P}(Q) \times \Sigma^* \rightarrow \mathcal{P}(Q)$ .

$$\hat{\delta}(B, w) = \begin{cases} B, & w = \varepsilon \\ \tilde{\delta}(\hat{\delta}(B, x), a), & w = xa, a \in \Sigma, x \in \Sigma^* \end{cases}$$

**Definicja 15.** Niech  $A = (Q, \Sigma, \delta, S, F)$  będzie NFA. Językiem akceptowanym przez  $A$  nazywamy język  $L(A) = \{w \in \Sigma^* : \hat{\delta}(S, w) \cap F \neq \emptyset\}$ .

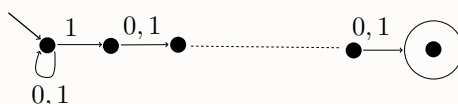
**Twierdzenie 2.** Niech  $L \subseteq \Sigma^*$ . Mamy  $L = L(A_D)$  dla pewnego DFA  $A_D$  wtedy i tylko wtedy, gdy  $L = L(A_N)$  dla pewnego NFA  $A_N$ .

**Dowód.** ( $\Rightarrow$ ) Jeśli DFA  $(Q, \Sigma, \delta, s, F)$  akceptuje  $L$ , to NFA  $(Q, \Sigma, \delta, \{s\}, F)$  również (każdy DFA jest NFA z minimalną różnicą typu krotki).

( $\Leftarrow$ ) Konstrukcja potęgowa – jeśli  $A_N = (Q, \Sigma, \delta, S, F)$  akceptuje  $L$ , to akceptuje je też DFA  $(\mathcal{P}(Q), \Sigma, \delta_D, S, \mathcal{F})$ , gdzie  $\delta_D = \tilde{\delta}$  (czyli  $\hat{\delta}_D = \hat{\delta}$ ) oraz  $\mathcal{F} = \{B \in \mathcal{P}(Q) : B \cap F \neq \emptyset\}$ .  $\square$

**Twierdzenie 3.** Dla każdego  $n \in \mathbb{N}_+$  istnieje NFA  $A_n$  o  $n + 1$  stanach taki, że DFA akceptujący  $L(A_n)$  ma co najmniej  $2^n$  stanów.

**Dowód.** Ustalmy alfabet  $\Sigma = \{0, 1\}$ . Niech  $A_n$  będzie NFA akceptującym język, w którym  $n$ -ta litera od końca to 1. Jest on zadany grafem jak na Rysunku 2 i ma dokładnie  $n + 1$  stanów.



Rysunek 2: Graf omawianego NFA. Strzałka bez litery oznacza stan początkowy.

Założmy, że istnieje DFA  $A_D$  taki, że  $L(A_D) = L(A_n)$  oraz liczba jego stanów jest ostro mniejsza od  $2^n$ .  $n$ -bitowych ciągów jest  $2^n$ , a więc muszą istnieć takie dwa różne ciągi  $\bar{a}, \bar{b} \in \{0, 1\}^n$ , którym  $A_D$  przypisuje ten sam stan. Niech  $i$  będzie pewnym indeksem takim, że  $a_i \neq b_i$ . Bez straty ogólności  $a_i = 1$  i  $b_i = 0$ . Rozważmy ciągi  $\bar{a}', \bar{b}'$  powstałe z poprzednich ciągów poprzez dopisanie do nich  $i$  zer. Takim ciągom  $A_D$  również przypisze ten sam stan (bo przypisze ich prefiksom, a potem nie mogą się rozejść, bo automat jest deterministyczny). Do tego litera, na której się różnią, jest  $n$ -ta od końca. Zatem  $\bar{a}' \in L(A_n)$  i  $\bar{b}' \notin L(A_n)$  – sprzeczność, bo  $A_D$  daje im ten sam stan.  $\square$

**Uwaga.** Widzimy, że DFA i NFA są tak samo mocne – akceptują dokładnie takie same rodziny języków. Jednak DFA wymaga wykładniczo więcej stanów.

**Definicja 16.** Definiujemy  $\varepsilon$ -NFA tak samo jak NFA, z tym, że relacja przejścia ma sygnaturę  $Q \times (\Sigma \cup \{\varepsilon\}) \times Q$  (zakładamy, że  $\varepsilon \notin \Sigma$ ). Oznacza to, że  $\varepsilon$ -NFA może wykonywać przejścia na słowie pustym, czyli zmienić stan bez „zużycia” litery z wejścia.

Funkcję  $\tilde{\delta}$  definiujemy identycznie jak dla NFA.

**Definicja 17.** Definiujemy  $\varepsilon$ -domknięcie zbioru stanów  $B$  w automacie  $A$  jako zbiór  $B^{A, \varepsilon}$  taki, że

$$B^{A, \varepsilon} = \bigcup_{i=0}^{\infty} B_i^{A, \varepsilon},$$

gdzie

$$B_i^{A,\varepsilon} = \begin{cases} B, & i = 0 \\ \tilde{\delta}(B_{i-1}^{A,\varepsilon}), & i > 0 \end{cases}.$$

Intuicyjnie, są to wszystkie stany, do których da się dojść z  $B$  za pomocą przejść  $\varepsilon$ .

**Definicja 18.** Analogicznie jak przedtem dla  $\varepsilon$ -NFA  $A$  definiujemy funkcję  $\hat{\delta} : \mathcal{P}(Q) \times \Sigma^* \rightarrow \mathcal{P}(Q)$ .

$$\hat{\delta}(B, w) = \begin{cases} B^{A,\varepsilon}, & w = \varepsilon \\ \tilde{\delta}(\hat{\delta}(B, x), a)^{A,\varepsilon}, & w = xa, a \in \Sigma, x \in \Sigma^* \end{cases}.$$

Jest to taka sama funkcja jak dla NFA z tym, że po zużyciu każdej litery dokonujemy  $\varepsilon$ -domknięcia. Język akceptowany przez  $\varepsilon$ -NFA definiujemy identycznie jak dla zwykłego NFA.

**Twierdzenie 4.** Niech  $L \subseteq \Sigma^*$ . Mamy  $L = L(A_D)$  dla pewnego DFA  $A_D$  wtedy i tylko wtedy, gdy  $L = L(A_N)$  dla pewnego  $\varepsilon$ -NFA  $A_N$ .

**Dowód.** ( $\implies$ ) Podobnie jak w przypadku NFA, każde DFA jest NFA, które z kolei jest  $\varepsilon$ -NFA.

( $\impliedby$ ) Jeśli  $A_N = (Q, \Sigma, \delta, S, F)$  akceptuje  $L$ , to akceptuje je też DFA  $(\mathcal{P}(Q), \Sigma, \delta_D, S^{A_N, \varepsilon}, \mathcal{F})$ , gdzie  $\delta_D = \tilde{\delta}^{A_N, \varepsilon}$  (czyli  $\hat{\delta}_D = \hat{\delta}$ ) oraz  $\mathcal{F} = \{B \in \mathcal{P}(Q) : B \cap F \neq \emptyset\}$ .  $\square$

### 3. Wyrażenia regularne

2025-02-22

**Definicja 19.** Wyrażenia regularne nad  $\Sigma$  to najmniejszy język  $\text{RE}_\Sigma$  nad alfabetem  $\Sigma' = \Sigma \cup \{+, \cdot, *, 0, 1, (, )\}$  taki, że

- $0, 1$  oraz wszystkie  $a \in \Sigma$  są elementami  $\text{RE}_\Sigma$ .
- jeśli  $\alpha, \alpha_1, \alpha_2 \in \text{RE}_\Sigma$ , to również  $\alpha_1 + \alpha_2, \alpha_1 \cdot \alpha_2, \alpha^*, (\alpha) \in \text{RE}_\Sigma$ .

**Definicja 20.** Interpretacją wyrażeń regularnych jest funkcja  $L : \text{RE}_\Sigma \rightarrow \mathcal{P}(\Sigma^*)$  zadana przez

- $L(0) = \emptyset, L(1) = \{\varepsilon\}, \forall a \in \Sigma \ L(a) = \{a\}$ .
- $L(\alpha + \beta) = L(\alpha) \cup L(\beta)$ .
- $L(\alpha \cdot \beta) = L(\alpha) \cdot L(\beta)$ .
- $L(\alpha^*) = (L(\alpha))^*$ .
- $L((\alpha)) = L(\alpha)$ .

Intuicyjnie wyrażenia regularne określają język – budujemy słowa za pomocą konkatenuowania, sumowania i wielokrotnego powtarzania bloków, gdzie blokami atomowymi są pojedyncze litery.

Kolejność wykonywania operacji ustalają nawiasy. W przeciwnym wypadku wygląda tak: najpierw gwiazdka Kleene'ego  $*$ , potem konkatenuacja  $\cdot$ , na koniec suma  $+$ .

**Twierdzenie 5.** Niech  $L \subseteq \Sigma^*$ . Mamy  $L = L(R)$  dla pewnego wyrażenia regularnego  $R$  wtedy i tylko wtedy, gdy  $L = L(A)$  dla pewnego DFA  $A$  (lub równoważnie NFA,  $\varepsilon$ -NFA).

**Dowód.** ( $\implies$ ) Stosując indukcję po długości wyrażenia regularnego wykazemy, że istnieje  $\varepsilon$ -NFA, które akceptuje ten sam język. Zaczniemy od przypadków bazowych. Jeśli  $R = 0, 1, a$ , to konstruujemy automat, który odpowiednio: nie ma stanów końcowych, akceptuje tylko słowo puste, akceptuje tylko zadaną literę.

Teraz przechodzimy do kroku indukcyjnego:

- Jeśli  $R = \alpha_1 + \alpha_2$ , to istnieją automaty DFA  $A_1, A_2$  takie, że  $L(A_1) = L(\alpha_1), L(A_2) = L(\alpha_2)$ . Tworzymy automat  $A_3$ , który zawiera stany, przejścia i stany końcowe jak w  $A_1$  i  $A_2$ , a do



tego dodatkowy stan  $q$  – jest to jedyny stan startowy i ma on  $\varepsilon$ -przejścia do stanów startowych  $A_1$  i  $A_2$ . Widać, że  $A_3$  akceptuje odpowiedni język.

- Jeśli  $R = \alpha_1 \cdot \alpha_2$ , to ponownie rozważamy automaty  $A_1, A_2$  i tworzymy  $A_3$  – mamy stan startowy  $q$  z  $\varepsilon$ -przejściem do stanu startowego  $A_1$ . Mamy też  $\varepsilon$ -przejścia ze stanów końcowych  $A_1$  do stanu startowego  $A_2$ . Automat  $A_3$  akceptuje tylko stany końcowe  $A_2$  – w ten sposób będziemy akceptować tylko słowa będące zlepieniem odpowiednich słów.
- Jeśli  $R = \alpha^*$ , to istnieje automat DFA  $A$  taki, że  $L(A) = L(\alpha)$ . Tworzymy automat  $B$ , który ma wszystkie stany i przejścia z  $A$  oraz stan startowy  $q$  z  $\varepsilon$ -przejściem do stanu startowego  $A$ . Każdy stan końcowy  $A$  ma  $\varepsilon$ -przejście do  $q$  i  $q$  też jest stanem końcowym – w ten sposób nasz automat będzie akceptował wszystkie słowa będące powtórzeniem słów z  $L(\alpha)$ .

( $\Leftarrow$ ) Niech DFA  $A = (Q, \Sigma, \delta, s, F)$ . Wprowadźmy porządek na stanach  $Q = \{q_1, \dots, q_n\}$ . Zauważmy, że  $w \in L(A)$  jest równoważne istnieniu ścieżki na stanach  $s \rightsquigarrow q_j \in F$ , którą automat przechodzi dla tego słowa. Skonstruujemy wyrażenie regularne, które generuje dokładnie takie słowa.

Niech  $R_{i,j}^{(k)}$  będzie wyrażeniem regularnym, którego językiem są słowa, które generują w  $A$  ścieżkę od stanu  $q_i$  do  $q_j$  i korzystają pomiędzy tylko ze stanów  $\{q_1, \dots, q_k\}$ . Zdefiniujemy takie wyrażenie indukcyjnie. Mamy

$$R_{i,j}^{(0)} = \beta_{i,j} + \sum_{a \in \Sigma: \delta(q_i, a) = q_j} a,$$

$$\text{gdzie } \beta_{i,j} = \begin{cases} 0, & i \neq j \\ 1, & i = j \end{cases}.$$

Teraz dla  $0 < k \leq n$  możemy zdefiniować  $R_{i,j}^{(k)} = R_{i,j}^{(k-1)} + R_{i,k}^{(k-1)} \left(R_{k,k}^{(k-1)}\right)^* R_{k,j}^{(k-1)}$  – mając do dyspozycji nowy stan  $q_k$  możemy albo znaleźć ścieżkę bez niego, albo dojść do niego korzystając tylko z mniejszych stanów, potem wrócić do niego dowolną liczbę razy i ostatecznie wyjść z niego i dojść do końca pozostałymi stanami.

Ostatecznie szukane wyrażenie to  $R = \sum_{q_j \in F} R_{i,j}^{(n)}$ , gdzie  $q_i = s$ .  $\square$

**Definicja 21.** Język, który jest akceptowany przez jakieś wyrażenie regularne (lub – równoważnie – skończony automat) nazywamy językiem regularnym.

**Definicja 22.** Wyrażenia regularne  $\alpha_1, \alpha_2$  nazywamy równoważnymi, jeśli  $L(\alpha_1) = L(\alpha_2)$ . Oznaczamy to  $\alpha_1 = \alpha_2$ .

**Propozycja 4.** Dla wyrażeń regularnych  $L, M, N$  zachodzi:

1.  $L + M = M + L$ .
2.  $(L + M) + N = L + (M + N)$ .
3.  $(LM)N = L(MN)$ .
4.  $0 + L = L$ .
5.  $1L = L1 = L$ .
6.  $0L = L0 = 0$ .
7.  $L(M + N) = LM + LN$ .
8.  $(M + N)L = ML + NL$ .
9.  $L + L = L$ .

## 4. Języki regularne

2025-02-23

**Lemat 3 (O pompowaniu).** Jeśli  $L$  jest językiem regularnym, to istnieje takie  $n > 0$  (zależne od  $L$ ), że dla każdego  $w \in L$  z  $|w| \geq n$  istnieje postać  $w = xyz$  taka, że  $|xy| \leq n$ ,  $|y| \geq 1$  i  $\forall i \in \mathbb{N} \ xy^i z \in L$ . Inaczej mówiąc: musi istnieć takie niepuste  $y$  będące blisko początku słowa, że można je „napompować” – powtórzyć dowolną liczbę razy.

**Dowód.** Niech  $A = (Q, \Sigma, \delta, s, F)$  będzie DFA akceptującym  $L$ . Ustalmy  $n = |Q|$  i weźmy dowolne słowo  $w \in L$  takie, że  $m = |w| \geq n$ . Niech  $w = a_0 \dots a_{m-1}$ .

Słowo jest akceptowane, a więc istnieje ciąg stanów  $q_0, \dots, q_m$  taki, że  $q_0 = s$  i  $q_m \in F$ . Jego długość jest większa niż  $n$ , a więc jakiś stan musi się powtarzać. Niech  $q_i$  będzie tym stanem, którego pierwsze powtórzenie występuje najwcześniej – niech będzie to  $q_j$ . Ustalmy

$$x = a_0 \dots a_{i-1}, \quad y = a_i \dots a_{j-1}, \quad z = a_j \dots a_{m-1}.$$

Mamy  $|xy| \leq n$ , bo inaczej  $q_i$  nie powtarzałby się najwcześniej. Do tego z  $q_i = q_j$  wynika, że słowo  $y$  może wystąpić w akceptowanym słowie dowolną liczbę razy – mamy cykl stanów. Zatem  $\forall i \in \mathbb{N} \ xy^i z \in L$ . Oczywiście  $y$  jest niepuste.  $\square$

**Twierdzenie 6.** Język

$$L = \{a^n b^n : n \in \mathbb{N}\}$$

nie jest regularny.

**Dowód.** Pokażemy, że  $L$  nie spełnia lematu o pompowaniu. Ustalmy dowolne  $n > 0$  i rozważmy słowo  $w = a^{2n} b^{2n}$ . Dla każdego podziału  $w = xyz$  takiego, że  $|xy| \leq n$  zachodzi  $xy = a^k$  dla pewnego  $k$ . Wobec tego mamy  $y = a^l$  dla pewnego  $l > 0$ . Gdyby lemat o pompowaniu był spełniony, to  $a^{2n-l} b^{2n}$  musiałoby należeć do języka, a tak nie jest. Zatem  $L$  nie spełnia lematu o pompowaniu i nie jest regularny.  $\square$

**Definicja 23.** Odwróceniem słowa  $w = a_1 \dots a_n$  nazywamy słowo  $w^R = a_n \dots a_1$ . Odwróceniem języka  $L$  nazywamy język  $L^R = \{w^R : w \in L\}$ .

**Definicja 24.** Homomorfizmem słów nazywamy funkcję  $h : \Sigma^* \rightarrow \Sigma'^*$ , która dla zadanego słowa  $w$  zamienia każdą jego literę na ustalone słowo: dla  $w = a_1 \dots a_n$  mamy  $h(w) = h(a_1) \dots h(a_n)$ .

Homomorfizm możemy też zastosować na języku:  $h(L) = \{h(w) : w \in L\}$ .

**Twierdzenie 7.** Języki regularne są zamknięte na

1. sumę, konkatenację i gwiazdkę Kleene’ego.
2. przecięcie, dopełnienie i różnicę.
3. odwrócenie.

**Dowód.** Pierwszy punkt wynika z definicji wyrażeń regularnych.

Rozważmy DFA  $A_1 = (Q_1, \Sigma, \delta_1, s_1, F_1)$ ,  $A_2 = (Q_2, \Sigma, \delta_2, s_2, F_2)$ . Zdefiniujmy DFA

$$B = (Q_1 \times Q_2, \Sigma, \delta', (s_1, s_2), F_1 \times F_2),$$

gdzie  $\delta'((q_1, q_2), a) = (\delta_1(q_1, a), \delta_2(q_2, a))$ . Jest to automat, który symuluje chodzenie w obu automatach na raz. Widać, że akceptuje dokładnie te słowa, które akceptują oba automaty  $A_1, A_2$ . Zatem  $L(B) = L(A_1) \cap L(A_2)$ , czyli języki regularne są zamknięte na branie przecięć.

Dla DFA  $A = (Q, \Sigma, \delta, s, F)$  automat  $B = (Q, \Sigma, \delta, s, F^c)$  akceptuje dokładnie te słowa, których nie akceptuje  $A$ . Zatem  $L(B) = L(A)^c$ , czyli języki regularne są zamknięte na branie przecięć.

Zamknięcie na różnicę wynika z równości  $L \setminus M = L \cap M^c$ .

Zamknięcie na odwrócenie pokazujemy indukcją po długości wyrażenia regularnego. Jeśli język  $L$  jest generowany przez wyrażenie regularne  $E$ , to konstruujemy  $E^R$  spełniające  $L(E^R) = L^R$ :

- jeśli  $E \in \{0, 1, a\}$ , to  $L = L^R$  i  $E^R = E$ .

- jeśli  $E = E_1 + E_2$ , to  $E^R = E_1^R + E_2^R$ .
- jeśli  $E = E_1 E_2$ , to  $E^R = E_2^R E_1^R$ .
- jeśli  $E = E_1^*$ , to  $E^R = (E_1^R)^*$ .

□

**Twierdzenie 8.** Jeśli  $h : \Sigma^* \rightarrow \Sigma'^*$  jest homomorfizmem, to dla języka regularnego  $L \subseteq \Sigma^*$  język  $h(L) \subseteq \Sigma'^*$  też jest regularny. Podobnie dla regularnego  $L' \subseteq \Sigma'^*$  język  $h^{-1}(L') \subseteq \Sigma^*$  też jest regularny.

**Dowód.** Dla wyrażenia regularnego  $R$  oznaczmy przez  $h(R)$  wyrażenie regularne, w którym każde wystąpienie  $a \in \Sigma$  zostaje zastąpione przez  $h(a) \in \text{RE}_{\Sigma'}$  (jest to poprawne wyrażenie regularne, bo jest konkatenacją poprawnych wyrażeń regularnych). Mamy  $L(h(R)) = h(L(R))$ , co dowodzimy indukcją po długości wyrażenia regularnego. Jeśli  $R \in \{0, 1, a\}$ , to homomorfizm robi dokładnie to, co chcemy. Jeśli  $R = R_1 + R_2$ , to

$$L(h(R)) = L(h(R_1) + h(R_2)) = L(h(R_1)) \cup L(h(R_2)) = h(L(R_1)) \cup h(L(R_2)) = h(L(R)),$$

gdzie pierwsze przejście wynika z tego, że  $h$  zmienia tylko litery. Pozostałe przypadki wyglądają podobnie. Zatem dla  $L = L(R)$  znaleźliśmy wyrażenie regularne akceptujące język  $h(L)$ .

Dla dowodu drugiej części rozważmy DFA  $A = (Q, \Sigma', \delta, s, F)$  akceptujące język  $L'$ . Zdefiniujmy DFA  $B = (Q, \Sigma, \gamma, s, F)$ , gdzie  $\gamma(q, a) = \hat{\delta}(q, h(a))$ .  $B$  symuluje zachowanie  $A$  na literach z  $\Sigma$  – przepuszcza je przez homomorfizm i zachowuje się jak  $A$  na otrzymanym słowie. Łatwo zauważyć, że  $\hat{\gamma}(s, w) = \hat{\delta}(s, h(w))$  (indukcja po długości słowa). Zatem  $B$  akceptuje słowo  $w$  wtedy i tylko wtedy, gdy  $A$  akceptuje  $h(w)$  – zatem  $L(B) = h^{-1}(L')$ . □

**Definicja 25.** Dla ustalonego DFA  $A = (Q, \Sigma, \delta, s, F)$  mówimy, że jego stany  $q_1, q_2$  są  $A$ -równoważne, jeśli

$$\forall w \in \Sigma^* \quad \hat{\delta}(q_1, w) \in F \iff \hat{\delta}(q_2, w) \in F.$$

Jeśli stany nie są  $A$ -równoważne, to są  $A$ -rozróżnialne, czyli

$$\exists w \in \Sigma^* \quad \hat{\delta}(q_1, w) \in F \iff \hat{\delta}(q_2, w) \notin F.$$

**Propozycja 5.**  $A$ -równoważność jest relacją równoważności.

**Notacja.** Dla DFA  $A$  i stanu  $q$  przez  $[q]_A$  oznaczamy klasę  $A$ -równoważności, do której należy  $q$ .

**Algorytm 1.** Dla zadanego DFA  $A$  rozważmy następujący algorytm: tworzymy macierz Boolowską  $M$ , w której  $M_{ij} = 1 \iff q_i, q_j$  są  $A$ -rozróżnialne. Robimy to zaczynając od macierzy zerowej i ustalając  $M_{ij} = 1$  dla  $q_i \in F, q_j \notin F$ .

Krok algorytmu wygląda tak: dopóki macierz się zmienia dla każdych dwóch stanów  $q_i, q_j$  i każdej litery  $a$ , jeśli  $\delta(q_i, a) = q_k$  i  $\delta(q_j, a) = q_l$  oraz  $M_{kl} = 1$ , to ustalamy  $M_{ij} = 1$ .

Tak zadany algorytm zwraca poprawny wynik.

**Dowód.** Ten algorytm działa mniej więcej jak Floyd-Warshall, zaczynamy od stanów które rozróżnia puste słowo i budujemy dalej – jeśli jakaś litera prowadzi do stanów, które są rozróżniane przez krótsze słowo, to można do niego dopisać tę literę i znaleźć słowo rozróżniające nasze stany.

Jeśli mamy  $M_{ij} = 1$ , to  $q_i$  i  $q_j$  są  $A$ -rozróżnialne, bo algorytm skonstruował słowo, które je rozróżnia. Załóżmy nie wprost, że stany  $q_i, q_j$  są rozróżnialne, a algorytm stwierdził  $M_{ij} = 0$ . Do tego słowo  $w$  świadczące o rozróżnialności jest najkrótszym, dla którego ta sytuacja może zajść. Jeśli  $w = \varepsilon$ , to z bazy algorytmu te stany będą rozróżnione. Zatem  $w = av$  i mamy

$$\left( \hat{\delta}(q_1, w) \in F \iff \hat{\delta}(q_2, w) \notin F \right) \iff \left( \hat{\delta}(\delta(q_1, a), v) \in F \iff \hat{\delta}(\delta(q_2, a), v) \notin F \right).$$

Stany  $q_k = \delta(q_1, a)$  i  $q_l = \delta(q_2, a)$  są rozróżniane przez  $v$ , a więc  $M_{kl} = 1$ , bo  $|v| < |w|$ . Widać, że

gdy algorytm ustali  $M_{kl} = 1$ , to w następnym kroku stwierdzi  $M_{ij} = 1$  – sprzeczność.  $\square$

**Twierdzenie 9.** Rozważmy DFA  $A = (Q, \Sigma, \delta, s, F)$ . Skonstruujmy automat  $D$  kolapsując stany  $A$ , które są  $A$ -równoważne – mamy  $D = (Q, \Sigma, \delta_D, [s]_A, \mathcal{F})$ , gdzie  $\mathcal{Q}$  i  $\mathcal{F}$  oznaczają rodziny klas  $A$ -równoważności, do których należą elementy odpowiednio  $Q$  i  $F$ , a  $\delta_D$  jest relacją  $\mathcal{Q} \times \Sigma \times \mathcal{Q}$ , w której

$$([q]_A, a, [g]_A) \in \delta_D \iff \exists_{q_1, q_2 \in Q} q_1 \in [q]_A \wedge q_2 \in [g]_A \wedge \delta(q_1, a) = q_2.$$

Następnie zmodyfikujmy  $D$ , usuwając stany, do których nie da się dojść ze stanu startowego. W takiej sytuacji  $D$  jest DFA i do tego jest najmniejszym na liczbę stanów DFA akceptującym ten sam język, co  $A$ .

**Dowód.**  $D$  mógłby nie być DFA, gdyby istniały jakieś  $A$ -równoważne stany  $q_1, q_2$  i litera  $a$  takie, że  $q_1$  przechodzi z nią do  $q_3$  a  $q_2$  do  $q_4$  i  $q_3$  oraz  $q_4$  są  $A$ -rozdzielalne. Wtedy jednak słowo  $a$  świadczyłoby o  $A$ -rozdzielalności  $q_1$  i  $q_2$ , co daje sprzeczność. Jasnym jest, że  $D$  akceptuje ten sam język, co  $A$ . Pozostało wykazać jego minimalność.

Założmy, że istnieje DFA  $B = (Q_B, \Sigma, \delta_B, s_B, F_B)$  takie, że  $L(B) = L(A)$  i  $B$  ma mniej stanów niż  $D$ . Zdefiniujmy DFA  $C = (Q \cup Q_B, \Sigma, \delta_D \cup \delta_B, s_B, \mathcal{F} \cup F_B)$ . Zauważmy, że dla każdego stanu  $q_D \in Q$  istnieje  $q_B \in Q_B$  taki, że  $q_D$  i  $q_B$  są  $C$ -równoważne –  $D$  i  $B$  akceptują te same języki, a więc na pewno  $[s]_A$  i  $s_B$  są  $C$ -równoważne (gdyby nie były, to jakieś słowo różniłoby te stany, a więc i języki). Z tego wynika, że dla każdego  $a \in \Sigma$  stany  $\delta([s]_A, a)$  i  $\delta(s_B, a)$  są  $C$ -równoważne (i tak dalej dla dłuższych słów).

Zatem każdy stan  $D$  (jako osiągalny ze stanu startowego) ma stan  $B$ , któremu jest  $C$ -równoważny. Stanów  $B$  jest mniej, a więc istnieją takie stany  $q_1, q_2 \in Q$ , że oba są równoważne pewnemu  $q_3 \in Q_B$ .  $C$ -równoważność jest przechodnia, a więc  $q_1$  i  $q_2$  są  $C$ -równoważne, czyli też  $D$ -równoważne. To jednak jest niemożliwe, bo w naszej konstrukcji skłapowaliśmy wszystkie równoważne stany w jeden. Uzyskana sprzeczność kończy dowód.  $\square$

**Uwaga.** Powyższe twierdzenie i algorytm dają nam metodę konstrukcji najmniejszego DFA akceptującego dany język.

**Definicja 26.** Mówimy, że relacja  $\equiv$  jest  $L$ -kongruentna dla pewnego języka  $L \subseteq \Sigma^*$ , jeśli

1.  $\forall_{v \in \Sigma^*} x \equiv y \implies xv \equiv yv$ .
2.  $x \equiv y \implies (x \in L \iff y \in L)$ .

**Twierdzenie 10 (Myhill, Nerode).** Język  $L$  jest regularny wtedy i tylko wtedy, gdy istnieje relacja równoważności  $\equiv \subseteq \Sigma^* \times \Sigma^*$  o skończenie wielu klasach równoważności, która jest  $L$ -kongruentna.

**Dowód.** ( $\implies$ ) Regularny język  $L$  ma swoje DFA  $A$ , na podstawie którego definiujemy

$$x \equiv y \iff \hat{\delta}(s, x) = \hat{\delta}(s, y).$$

Ta relacja jest równoważnością, a do tego dla każdego  $v \in \Sigma^*$  i dowolnych  $x, y \in \Sigma^*$  zachodzi

$$x \equiv y \iff \hat{\delta}(s, x) = \hat{\delta}(s, y) \implies \hat{\delta}(s, xv) = \hat{\delta}(s, yv) \iff xv \equiv yv,$$

$$x \equiv y \iff \hat{\delta}(s, x) = \hat{\delta}(s, y) \implies (x \in L \iff y \in L).$$

Zatem nasza relacja jest  $L$ -kongruentna. Do tego ma skończenie wiele klas równoważności, bo co najwyżej tyle, ile  $A$  ma stanów.

( $\impliedby$ ) Tworzymy automat, w którym stany to klasy równoważności naszej relacji  $\equiv$  (tutaj ważna jest ich skończona ilość). Jako stany końcowe przyjmujemy te klasy, których elementy należą do języka (drugi warunek z definicji  $L$ -kongruentności). Stanem początkowym jest klasa, do której należy  $\varepsilon$ . Funkcję przejścia definiujemy w następujący sposób: dla zadanej klasy i litery konkatenujemy tę literę do dowolnego słowa z tej klasy i tworzymy przejście do klasy, do której należy wynik. Z pierwszego warunku z definicji  $L$ -kongruentności będzie to poprawna funkcja. W ten sposób stworzyliśmy DFA.

Zauważmy, że  $\hat{\delta}(s, w)$  jest klasą równoważności, do której należy  $w$  – dowód indukcyjną po długości  $w$ , dla słowa pustego wszystko się zgadza z definicji stanu początkowego. Dla  $w' = wa$  wiemy z założenia indukcyjnego, że  $\hat{\delta}(s, w)$  jest klasą, do której należy  $w$ . Przejście po literze  $a$  powoduje przejście do klasy  $wa$ , bo tak zdefiniowaliśmy  $\delta$ .

Z tego mamy  $L(A) = L$ , bo  $A$  akceptuje dokładnie te słowa, których klasy równoważności zawierają się w  $L$ .  $\square$

## 5. Problemy decyzyjne

2025-02-24

**Pomysł.** Dla zadanej reprezentacji języka chcemy znaleźć algorytmy, które stwierdzają różne rzeczy o tym języku.

**Uwaga.** Rozwiązując jakiś problem decyzyjny dla języka regularnego najlepiej mówić o postaci  $\varepsilon$ -NFA: NFA i DFA to jej szczególne przypadki, a wyrażenie regularne da się liniowo przekształcić do  $\varepsilon$ -NFA.

**Definicja 27.** Mając zadany język  $L$  i słowo  $v$  problem MEMBERSHIP polega na stwierdzeniu, czy  $v \in L$ .

**Twierdzenie 11.** Dla języków regularnych problem MEMBERSHIP jest wielomianowy.

**Dowód.** Pamiętamy zbiór stanów osiągalnych dla kolejnych prefiksów słowa – zaczynamy od  $\varepsilon$ -dopełnionych stanów początkowych, następnie przechodzimy kolejnymi literami do następnych stanów (i bierzemy  $\varepsilon$ -dopełnienia). Na koniec sprawdzamy, czy któryś z ostatecznych stanów jest stanem końcowym. Każdy z tych kroków działa wielomianowo.  $\square$

**Definicja 28.** Mając zadany język  $L$  problem EMPTINESS polega na stwierdzeniu, czy  $L = \emptyset$ .

**Twierdzenie 12.** Dla języków regularnych problem EMPTINESS jest wielomianowy.

**Dowód.** Wystarczy przejść po krawędziach grafu automatu i sprawdzić, czy da się dojść ze stanów początkowych do jakiegokolwiek stanu końcowego.

Dla wyrażenia regularnego można działać indukcyjnie: z bazowych wyrażeń tylko  $0$  jest puste,  $\alpha + \beta$  jest puste jak oba są puste,  $\alpha\beta$  jest puste jak któreś jest puste,  $\alpha^*$  nigdy nie jest puste.  $\square$

**Definicja 29.** Mając zadany język  $L$  problem INFINITENESS polega na stwierdzeniu, czy  $|L| = \aleph_0$ .

**Twierdzenie 13.** Dla języków regularnych problem INFINITENESS jest wielomianowy.

**Dowód.** Zauważmy, że język jest nieskończony wtedy i tylko wtedy, gdy w grafie jego automatu istnieje cykl (nie składający się z samych  $\varepsilon$ -przejsć), do którego da się dojść ze stanu początkowego i z którego da się dojść do stanu końcowego. Możemy zatem usunąć wszystkie nieosiągalne stany (znajdujemy, np. BFS'em ze stanów startowych), potem wszystkie stany, z których nie da się dojść do stanu końcowego i na koniec sprawdzić, czy w grafie istnieje odpowiedni cykl – jeśli  $\varepsilon$ -krawędzie dostaną wagę  $0$  a pozostałe  $-1$ , to algorytm Bellmana-Forda stwierdzi, czy istnieje ujemny cykl, a tego szukamy.

Dla wyrażenia regularnego można działać indukcyjnie: aby język był nieskończony na pewno potrzebna jest gwiazdka Kleene'ego, ale mamy  $0^* = 1$  i  $1^* = 1$ , zatem z przypadków bazowych tylko  $a^*$  jest nieskończony. Dla  $\alpha + \beta$  jedno musi być nieskończone, dla  $\alpha\beta$  jedno musi być nieskończone a drugie niepuste, dla  $\alpha^*$  pod gwiazdką musi być coś nierównego  $0, 1$ .  $\square$

**Definicja 30.** Mając zadane języki  $L_1, L_2$  problem EQUIVALENCE polega na stwierdzeniu, czy  $L_1 = L_2$ .

**Twierdzenie 14.** Dla języków regularnych problem EQUIVALENCE jest rozstrzygalny, a dla reprezentacji przez DFA jest do tego wielomianowy.

**Dowód.** Mając zadane DFA  $A_1, A_2$  takie, że  $L_1 = L(A_1), L_2 = L(A_2)$  można zauważyć, że

$$L(A_1) = L(A_2) \iff (L(A_1) \cap L(A_2)^c) \cup (L(A_1)^c \cap L(A_2)) = \emptyset.$$

Zatem wystarczy skonstruować dwa automaty dla dopełnienia i dwa dla przecięć (wszystko wielomianowo za pomocą wcześniej wprowadzonych konstrukcji), a następnie sprawdzić pustość obu języków generowanych przez automaty przecięć.

Dla innych reprezentacji wystarczy przekształcić je na DFA (być może wykładniczo) i zastosować ten algorytm.  $\square$

## 6. Gramatyki bezkontekstowe

2025-02-24

**Definicja 31.** Gramatyka bezkontekstowa (CFG, context-free grammar) to krotka  $G = (N, \Sigma, P, S)$ , gdzie:

- $N$  - skończony zbiór zmiennych (nieterminale),
- $\Sigma$  - skończony alfabet (terminale),
- $P \subseteq N \times (N \cup \Sigma)^*$  - zbiór produkcji,
- $S \in N$  - symbol startowy.

**Notacja.** Produkcję  $(s, w)$  będziemy zapisywać  $s \rightarrow w$ . Kilka produkcji  $s \rightarrow w_1, s \rightarrow w_2$  możemy zapisać łącznie:  $s \rightarrow w_1 \mid w_2$ .

**Definicja 32.** Forma zdaniowa to dowolne słowo z  $(N \cup \Sigma)^*$ .

**Definicja 33.** Dla gramatyki bezkontekstowej  $G = (N, \Sigma, P, S)$  definiujemy relację  $\rightarrow_G$  na formach zdaniowych. Mówimy, że  $\alpha \rightarrow_G \beta$  (możemy uzyskać  $\beta$  z  $\alpha$  w jednym kroku), jeśli

$$\exists_{\alpha_1, \alpha_2, \gamma \in (N \cup \Sigma)^*} \exists_{A \in N} (A, \gamma) \in P \wedge \alpha = \alpha_1 A \alpha_2 \wedge \beta = \alpha_1 \gamma \alpha_2,$$

czyli  $\beta$  powstaje z  $\alpha$  przez zamianę nieterminała na formę zdaniową zgodnie z jakąś produkcją.

**Przykład.** Język palindromów nad  $\{0, 1\}$  można zdefiniować rekurencyjnie:  $0, 1, \varepsilon$  są palindromami, dla palindromu  $P$  palindromami są  $0P0, 1P1$ .

Można to wypowiedzieć w języku gramatyk bezkontekstowych: dla  $G = (\{S\}, \{0, 1\}, P, S)$  do  $P$  należą produkcje  $S \rightarrow \varepsilon \mid 0 \mid 1$  i  $S \rightarrow 0S0 \mid 1S1$ .

**Definicja 34.** Przez  $\rightarrow_G^*$  oznaczamy zwrotne i przechodnie domknięcie  $\rightarrow_G$ , czyli  $\alpha \rightarrow_G^* \beta$ , jeśli możemy uzyskać  $\beta$  z  $\alpha$  w pewnej liczbie kroków.

**Definicja 35.** Język  $L(G)$  generowany przez gramatykę  $G = (N, \Sigma, P, S)$  to język

$$L(G) = \{w \in \Sigma^* : S \rightarrow_G^* w\}.$$

Ogólniej można zdefiniować

$$L(G, A) = \{w \in \Sigma^* : A \rightarrow_G^* w\}.$$

Takie języki nazywamy językami bezkontekstowymi (CFL).

**Definicja 36.** Wywód (derivation) w gramatyce  $G$  to ciąg form zdaniowych  $\alpha_0, \dots, \alpha_n$  takich, że  $\forall_{i < n} \alpha_i \rightarrow_G \alpha_{i+1}$ .

**Uwaga.**  $v \in L(G)$  wtedy i tylko wtedy, gdy istnieje wywód  $\alpha_0, \dots, \alpha_n$  w  $G$  taki, że  $\alpha_0 = S$  i  $\alpha_n = v$ .



**Definicja 37.** Wywód lewostronny  $\alpha_0, \dots, \alpha_n$  to taki wywód, że dla każdego  $i < n$  mamy

$$\alpha_i = xAB_i,$$

gdzie  $x \in \Sigma^*$ ,  $A \in N$ ,  $B_i \in (N \cup \Sigma)^*$ . Przez  $A$  rozumiemy nieterminal, który jest zastępowany w danym kroku wyvodu. Zatem w wywodzie lewostronnym zawsze zamieniamy najbardziej lewy nieterminal.

**Definicja 38.** Drzewo wyvodu (parse tree) formy zdaniowej  $\alpha$  w gramatyce  $G = (N, \Sigma, P, S)$  to ukorzenione drzewo z porządkiem na dzieciach, w którym każdy wierzchołek ma etykietę z  $N \cup \Sigma \cup \{\varepsilon\}$ . Etykieta korzenia to  $S$ . Jeśli wierzchołek ma etykietę  $A$  i dzieci  $X_1, \dots, X_n$ , to  $A \rightarrow X_1 \dots X_n$  jest produkcją w  $P$ . Wierzchołki o etykiecie  $\varepsilon$  są liśćmi i nie mają rodzeństwa. W wyniku konkatenacji etykiet liści otrzymujemy  $\alpha$ .

**Propozycja 6.** Każde drzewo wyvodu można jednoznacznie przypisać do wyvodu lewostronnego.

**Dowód.** Mając zadane drzewo wyvodu dla formy zdaniowej  $\alpha$  można przejść po nim zgodnie z kolejnością dzieci. W ten sposób z symbolu startowego dostajemy  $\alpha$ , a w kolejnych formach zdaniowych zmienia się najbardziej lewy nieterminal. Podobnie wywód lewostronny zadaje drzewo wyvodu – budujemy drzewo zgodnie z kolejnymi przejściami w wywodzie.  $\square$

**Definicja 39.** Gramatyka jest jednoznaczna (unambiguous), jeśli każde słowo  $v \in L(G)$  ma dokładnie jedno drzewo wyvodu (wywód lewostronny) w tej gramatyce.

**Definicja 40.** Język bezkontekstowy jest wewnętrznie niejednoznaczny (inherently ambiguous), jeśli każda generująca go gramatyka jest niejednoznaczna.

**Przykład.** Rozważmy język  $L_{\text{BAL}}$  nad  $\{(, )\}$  składający się z poprawnych nawiasowań. Generuje go gramatyka  $G_{\text{BAL}} = (\{B\}, \{(, )\}, P, B)$ , gdzie  $P$  zawiera  $B \rightarrow \varepsilon \mid (B) \mid BB$ . Nie jest ona jednoznaczna: dla słowa  $()()()$  mamy wywody

$$B \rightarrow BB \rightarrow (B)B \rightarrow ()B \rightarrow ()BB \rightarrow ()(B)B \rightarrow ()()B \rightarrow ()()(B) \rightarrow ()()()$$

$$B \rightarrow BB \rightarrow BBB \rightarrow (B)BB \rightarrow ()BB \rightarrow ()(B)B \rightarrow ()()B \rightarrow ()()(B) \rightarrow ()()()$$

Sam język jest jednak jednoznaczny, o czym świadczy gramatyka  $G'_{\text{BAL}} = (\{S\}, \{(, )\}, P, S)$ , gdzie  $P$  zawiera  $S \rightarrow S(S) \mid \varepsilon$ .

## 7. Postać normalna Chomsky'ego

2025-03-01

**Definicja 41.** Mówimy, że symbol  $X \in N \cup \Sigma$  jest generujący, jeśli  $X \rightarrow_G^* w$  dla pewnego  $w \in \Sigma^*$ .

**Definicja 42.** Mówimy, że symbol  $X \in N \cup \Sigma$  jest osiągalny, jeśli  $S \rightarrow_G^* \alpha X \beta$  dla pewnych form zdaniowych  $\alpha, \beta$ .

**Definicja 43.** Mówimy, że symbol  $X \in N \cup \Sigma$  jest użyteczny, jeśli jest osiągalny i generujący.

**Twierdzenie 15.** Niech  $G = (N, \Sigma, P, S)$  będzie CFG, dla której  $L(G) \neq \emptyset$ . Zdefiniujmy nową gramatykę  $G_1 = (N_1, \Sigma_1, P_1, S)$  powstałą w następujący sposób.

1. Usuwamy z  $G$  wszystkie symbole niegenerujące i produkcje je zawierające, otrzymując gramatykę  $G_2 = (N_2, \Sigma_2, P_2, S)$ .
2. W podobny sposób usuwamy symbole nieosiągalne w  $G_2$ .

Gramatyka  $G_1$  nie zawiera symboli bezużytecznych i jest  $L(G_1) = L(G)$ .

**Dowód.** Niech  $X \in N_1 \cup \Sigma_1$ . Wiemy, że  $X \rightarrow_G^* w$  dla pewnego  $w \in \Sigma^*$  (inaczej zostałby usunięty w pierwszym kroku jako niegenerujący). Wszystkie symbole użyte w tym wywodzie również są

generujące, a więc mamy  $X \rightarrow_{G_2}^* w$ . Podobnie  $X$  nie został usunięty w drugim kroku, a więc  $S \rightarrow_{G_2}^* \alpha X \beta$  dla pewnych form zdaniowych  $\alpha, \beta$ . Wszystkie symbole w tym wywodzie są osiągalne, a więc  $S \rightarrow_{G_1}^* \alpha X \beta$ .

Każdy symbol w  $\alpha X \beta$  jest osiągalny i jest w  $N_2 \cup \Sigma_2$ , a więc  $\alpha X \beta \rightarrow_{G_2}^* xwy$ . Wszystkie symbole w tym wywodzie są osiągalne, a więc również w  $G_1$  zachodzi  $S \rightarrow_{G_1}^* \alpha X \beta \rightarrow_{G_1}^* xwy$ , czyli  $X$  jest użyteczny. Zatem  $G_1$  nie ma bezużytecznych symboli.

Oczywiście mamy  $L(G_1) \subseteq L(G)$ , bo tylko usuwaliśmy symbole i produkcje.  $L(G) \subseteq L(G_1)$  wynika z tego, że jeśli  $S \rightarrow_G^* w$ , to każdy symbol w tym wywodzie jest użyteczny, a więc  $S \rightarrow_{G_1}^* w$ .  $\square$

**Przykład.** Usuwanie symboli bezużytecznych nie można wykonać w odwrotnej kolejności. Dla gramatyki z produkcjami  $S \rightarrow AB|a$ ,  $A \rightarrow b$  wszystkie symbole są osiągalne, a po usunięciu symboli niegenerujących dostajemy  $S \rightarrow a$ ,  $A \rightarrow b$ . Widzimy, że  $A, b$  są bezużyteczne.

**Algorytm 2.** Dla zadanej gramatyki  $G = (N, \Sigma, P, S)$  można wyznaczyć symbole generujące dynamicznie: bazowo uznajemy za generujące symbole z  $\Sigma$ , potem przechodzimy po produkcjach i uznajemy za generujące te nieterminale, dla których istnieje produkcja tworząca formę zdaniową składającą się tylko z symboli generujących (w szczególności pustą). Powtarzamy takie przejście dopóki coś się zmienia.

Symbole osiągalne wyznaczamy algorytmem podobnym do przejścia grafu: zaczynamy odwiedzając  $S$ , odwiedzamy wszystkie symbole, które da się wyprodukować z odwiedzonych symboli, powtarzamy, dopóki kolejne iteracje odwiedzają nowe symbole.

Pierwszy z tych algorytmów działa w czasie  $|\Sigma| + |P| |N|$ , a drugi w  $|P| |N \cup \Sigma|$ .

**Definicja 44.** Nieterminal  $A$  jest wymazujący (nullable), jeśli  $A \rightarrow_G^* \varepsilon$ . Symbol  $a \in \Sigma$  zawsze jest niewymazujący.

**Twierdzenie 16.** Niech  $G = (N, \Sigma, P, S)$  będzie CFG. Niech  $G' = (N', \Sigma, P', S)$  będzie gramatyką otrzymaną z  $G$  w następujący sposób. Dla każdej produkcji  $A \rightarrow X_1 \dots X_k$  w  $P$  do  $P'$  należą wszystkie takie produkcje  $A \rightarrow Y_1 \dots Y_l$ , że  $Y_1 \dots Y_l$  jest pewnym niepustym podciągiem  $X_1 \dots X_k$ , który zawiera wszystkie symbole niewymazujące i pewne wymazujące.

Gramatyka  $G'$  nie zawiera symboli wymazujących i jest  $L(G') = L(G) \setminus \{\varepsilon\}$ .

**Dowód.** W  $P'$  nie ma produkcji postaci  $A \rightarrow \varepsilon$ , a więc nie ma symboli wymazujących.

Zauważmy, że gramatyka  $G'$  „modeluje” symbole wymazujące poprzez dodanie wszystkich produkcji powstałych poprzez usunięcie dowolnego ich podzbioru z pewnej produkcji w  $P$ . Zatem jeśli  $S \rightarrow_G^* w$ , to dostajemy  $S \rightarrow_{G'}^* w$  przechodząc tymi produkcjami, dla których odpowiednie symbole wymazujące znikają. Podobnie w drugą stronę – wystarczy wybierać produkcje, z których powstały te użyte. Formalny dowód przebiega indukcyjną po długości wyvodu.  $\square$

**Algorytm 3.** Nieterminale wymazujące łatwo wyznaczyć algorytmem dynamicznym: bazowo generujące są takie  $A$ , że  $A \rightarrow \varepsilon$  jest produkcją, potem przechodzimy po produkcjach i uznajemy za wymazujące te nieterminale, dla których istnieje produkcja tworząca formę zdaniową składającą się tylko z nieterminali wymazujących. Powtarzamy takie przejście dopóki coś się zmienia. Taki algorytm działa w czasie  $|P| |N|$ .

**Definicja 45.** Produkcję nazywamy jednostkową, jeśli jest postaci  $A \rightarrow B$  dla  $A, B \in N$ .

**Twierdzenie 17.** Niech  $G = (N, \Sigma, P, S)$  będzie CFG. Niech  $G' = (N', \Sigma, P', S)$  będzie gramatyką otrzymaną z  $G$  w następujący sposób. Rozważmy zbiór  $J = \{(A, B) \in N^2 : A \rightarrow B \in P\}$  i domknijmy go zwrotnie i przechodnio do  $J^*$ . Następnie dla każdego  $(A, B) \in J^*$  umieszczamy w  $P'$  wszystkie produkcje  $A \rightarrow \alpha$ , gdzie  $B \rightarrow \alpha$  jest niejednostkową produkcją w  $P$ .

Gramatyka  $G'$  nie ma produkcji jednostkowych i jest  $L(G') = L(G)$ .



**Dowód.** Umieszczamy w  $P'$  tylko produkcje niejednostkowe, a więc nie ma żadnych jednostkowych. Języki się zgadzają, bo wywody w  $G'$  są tylko skróconymi wywodami z  $G$  – wykonujemy produkcje jednostkowe „w miejscu” poprzez dodanie ich ciągów jako osobnych produkcji.  $\square$

**Wniosek.** Dla każdej CFG  $G$  generującej coś więcej niż  $\varepsilon$  istnieje CFG  $G_1$  taka, że  $L(G_1) = L(G) \setminus \{\varepsilon\}$  oraz  $G_1$  nie zawiera produkcji jednostkowych, symboli wymazujących oraz symboli bezużytecznych.

**Dowód.** Korzystając z przedstawionych wcześniej metod możemy usunąć symbole wymazujące, następnie usunąć produkcje jednostkowe (co nie wprowadza nowych symboli wymazujących), a następnie usunąć symbole bezużyteczne (co tylko usuwa produkcje i symbole, więc nie wprowadzi rzeczy już usuniętych).  $\square$

**Definicja 46.** Gramatyka  $G = (N, \Sigma, P, S)$  jest w postaci normalnej Chomsky’ego (CNF), jeśli każda produkcja jest postaci  $A \rightarrow BC$  dla  $A, B, C \in N$  lub  $A \rightarrow a$  dla  $a \in \Sigma$ . Do tego wszystkie symbole są użyteczne.

**Twierdzenie 18.** Dla każdej CFG  $G$  istnieje CFG  $G'$  w postaci normalnej Chomsky’ego taka, że  $L(G) \setminus \{\varepsilon\} = L(G')$ .

**Dowód.** Jeśli  $G$  generuje  $\emptyset$  lub  $\{\varepsilon\}$  to twierdzenie jest oczywiste. Dalej zakładamy, że  $G$  nie zawiera produkcji jednostkowych, symboli wymazujących oraz symboli bezużytecznych i nie generuje  $\varepsilon$ , czyli każda produkcja jest postaci  $A \rightarrow \alpha_1 \dots \alpha_k$  dla  $k \geq 2$  i  $\alpha_i \in \Sigma \cup N$  lub  $A \rightarrow \alpha$  dla  $\alpha \in \Sigma$ .

Jeśli  $\alpha_i \in \Sigma$ , to wprowadzamy nieterminal  $B_i$  z produkcją  $B_i \rightarrow \alpha_i$ , a następnie zamieniamy wszystkie wystąpienia  $\alpha_i$  na  $B_i$ . Teraz problemem są tylko produkcje postaci  $A \rightarrow B_1 \dots B_k$  dla  $k \geq 3$ . Możemy wprowadzić nowe nieterminale  $C_1, \dots, C_{k-2}$  oraz produkcje  $C_i \rightarrow B_{i+1}C_{i+1}$  dla  $i \in [k-3]$  oraz  $C_{k-2} \rightarrow B_{k-1}B_k$ . Teraz wystarczy zastąpić problematyczną produkcję przez  $A \rightarrow B_1C_1$ .  $\square$

**Algorytm 4 (Cocke, Younger, Kasami (CYK)).** Mając zadaną gramatykę  $G = (N, \Sigma, P, S)$  w CNF i słowo  $w \in \Sigma^*$  chcemy stwierdzić, czy  $w \in L(G)$ .

Robimy to dynamicznie: niech  $dp[i, j]$  oznacza listę wszystkich nieterminali, które generują podsłowo  $w$  długości  $i$  o początku w  $j$ . Bazą jest  $i = 1$ , do list wsadzamy nieterminale generujące daną literkę (łatwo znaleźć z postaci Chomsky’ego). Następnie iterujemy się po rosnących  $i$ , obliczając kolejne listy – dzielimy interesujące nas podsłowo na dwa podsłowa (na wszystkie sposoby), znając nieterminale generujące pierwsze i drugie podsłowo znajdujemy te nieterminale, które generują takie dwa nieterminale w odpowiedniej kolejności (znów postać Chomsky’ego).

Na koniec sprawdzamy, czy całe słowo jest generowane przez nieterminal startowy. Taki algorytm działa w czasie  $O(|P||v|^3)$ .

## 8. Automaty ze stosem

2025-03-17

**Definicja 47.** Automat ze stosem (PDA – pushdown automaton) to krotka

$$P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F),$$

gdzie:

- $Q$  – skończony zbiór stanów,
- $\Sigma$  – skończony alfabet,
- $\Gamma$  – skończony alfabet stosowy,
- $\delta \subseteq Q \times (\Sigma \cup \{\varepsilon\}) \times \Gamma \times Q \times \Gamma^*$  – skończona relacja przejścia,
- $q_0 \in Q$  – stan startowy,

- $Z_0 \in \Gamma$  – stosowy symbol początkowy (sygnalizujący pusty stos),
- $F \subseteq Q$  – zbiór stanów końcowych.

Taki automat działa podobnie jak  $\varepsilon$ -NFA, ale tym razem ma stos, na który może odkładać różne wartości, które potem są z niego ściągane i uczestniczą w podejmowaniu decyzji.

**Przykład.** Zdefiniujemy PDA generujące język parzystych palindromów  $L = \{ww^R : w \in \{0,1\}^*\}$ . Ustalamy  $P = (\{q_0, q_1, q_2\}, \{0,1\}, \{0,1,Z_0\}, \delta, q_0, Z_0, \{q_2\})$ , gdzie  $\delta$  zawiera przejścia:

- $(q_0, 0, Z_0, q_0, 0Z_0), (q_0, 1, Z_0, q_0, 1Z_0)$  – przejścia początkowe
- $(q_0, 0, 0, q_0, 00), (q_0, 1, 0, q_0, 10), (q_0, 0, 1, q_0, 01), (q_0, 1, 1, q_0, 11)$  – dokładanie liter na stos
- $(q_0, \varepsilon, Z_0, q_1, Z_0), (q_0, \varepsilon, 0, q_1, 0), (q_0, \varepsilon, 1, q_1, 1)$  – przejście do drugiej części słowa
- $(q_1, 0, 0, q_1, \varepsilon), (q_1, 1, 1, q_1, \varepsilon)$  – ściąganie liter, jeśli się zgadzają
- $(q_1, \varepsilon, Z_0, q_2, Z_0)$  – akceptacja słowa.

Taki automat można też prezentować graficznie podobnie jak automaty skończone, ale trzeba dopisywać przy strzałkach jak wyglądają zmiany stosu.

**Definicja 48.** Opis chwilowy (konfiguracja) PDA to trójka  $(q, w, \gamma)$ , gdzie  $q$  to aktualny stan,  $w$  to nieprzetworzona jeszcze część słowa a  $\gamma$  to wszystkie symbole na stosie (czytane od góry).

**Definicja 49.** Dla PDA  $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$  definiujemy  $\vdash_P$  jako relację przejścia między konfiguracjami, to znaczy

$$(q, aw, X\beta) \vdash_P (p, w, \alpha\beta) \iff (q, a, X, p, \alpha) \in \delta,$$

gdzie  $a \in \Sigma \cup \{\varepsilon\}$ ,  $X \in \Gamma$  a pozostałe symbole mają typy wynikające z definicji.

Definiujemy  $\vdash_P^*$  jako zwrotne i przechodnie domknięcie  $\vdash_P$ .

**Propozycja 7.** Dla PDA  $P$  jeśli  $(q, x, \alpha) \vdash_P^* (p, y, \beta)$ , to również  $(q, xw, \alpha\gamma) \vdash_P^* (p, yw, \beta\gamma)$ .

**Dowód.** Założenie oznacza, że konsumując znaki z  $x$  i  $\alpha$  możemy dojść do odpowiedniej konfiguracji. Mając dołożone na ich koniec jakieś znaki  $w$  i  $\gamma$  możemy zastosować te same przejścia, by otrzymać odpowiednią konfigurację. Te przejścia oczywiście nie dotkną dołożonych znaków.  $\square$

**Propozycja 8.** Dla PDA  $P$  jeśli  $(q, xw, \alpha) \vdash_P^* (p, yw, \beta)$ , to również  $(q, x, \alpha) \vdash_P^* (p, y, \beta)$ .

**Dowód.** Jeśli w odpowiednich przejściach nie zaczęliśmy konsumować  $w$ , to równie dobrze można je usunąć z naszego słowa.  $\square$

**Definicja 50.** Niech  $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$  będzie PDA. Językiem akceptowanym stanem końcowym tego PDA nazywamy język  $L(P) = \{w \in \Sigma^* : (q_0, w, Z_0) \vdash_P^* (q, \varepsilon, \alpha) \wedge q \in F\}$ .

**Definicja 51.** Niech  $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$  będzie PDA. Językiem akceptowanym pustym stosom tego PDA nazywamy język  $N(P) = \{w \in \Sigma^* : (q_0, w, Z_0) \vdash_P^* (q, \varepsilon, \varepsilon)\}$ .

**Twierdzenie 19.** Mamy  $L = L(P)$  dla pewnego PDA  $P$  wtedy i tylko wtedy, gdy  $L = N(Q)$  dla pewnego PDA  $Q$ .

**Dowód.** ( $\implies$ ) Dla  $P = (Q, \Sigma, \Gamma, \delta, p_0, Z_0, F)$  niech  $Q = (Q \cup \{q_0, q_1\}, \Sigma, \Gamma \cup \{X_0\}, \delta_Q, q_0, X_0, F)$ . W  $\delta_Q$  mamy takie same przejścia jak w  $\delta$ , a do tego  $(q_0, \varepsilon, X_0, p_0, Z_0X_0), (q_1, \varepsilon, A, q_1, \varepsilon)$  dla każdego  $A \in \Gamma \cup \{X_0\}$  oraz  $(p_F, \varepsilon, A, q_1, \varepsilon)$  dla każdego  $A \in \Gamma$  i  $p_F \in F$ . Ten automat po przejściu do stanu akceptującego opróżni stos i jednocześnie dzięki symbolowi  $X_0$  nie opróżni go w żaden inny sposób.

( $\impliedby$ ) Dla  $Q = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$  definiujemy  $P = (Q \cup \{p_0, p_1\}, \Sigma, \Gamma \cup \{X_0\}, \delta_P, p_0, X_0, \{p_1\})$ . W  $\delta_P$  mamy takie same przejścia jak w  $\delta$ , a do tego  $(p_0, \varepsilon, X_0, q_0, Z_0X_0)$  i  $(q, \varepsilon, X_0, p_1, \varepsilon)$  dla każdego  $q \in Q$ . W ten sposób nasz automat włoży  $Z_0$  na stos, zachowa się dokładnie jak  $Q$  (czyli dokładnie

słowa akceptowane przez  $Q$  skończą ze stosem zawierającym samo  $X_0$ ), a następnie przejdzie do stanu akceptującego.  $\square$

**Twierdzenie 20.** Język  $L$  jest bezkontekstowy wtedy i tylko wtedy, gdy jest akceptowany przez pewien automat ze stosem.

**Dowód.** ( $\Rightarrow$ ) Język  $L$  jest generowany przez pewną gramatykę  $G'$ . Modyfikujemy ją do gramatyki  $G = (N, \Sigma, P, S)$ , w której każda produkcja jest postaci  $A \rightarrow a$  dla  $a \in \Sigma \cup \{\varepsilon\}$  lub  $A \rightarrow B_1 \dots B_n$  dla  $A, B_1, \dots, B_n \in N$ . Robimy to dodając produkcję  $A \rightarrow a$  dla każdego terminala  $a$  i zastępując go w każdej produkcji z  $G'$  przez odpowiedni nieterminal. Widać, że  $G$  akceptuje  $L$ . Konstruujemy PDA  $P = (\{q\}, \Sigma, N, \delta, q, S, \{q\})$ , które będzie akceptować stosem pustym. Dla produkcji  $A \rightarrow a$  dodajemy  $(q, a, A, q, \varepsilon) \in \delta$ , a dla produkcji  $A \rightarrow B_1 \dots B_n$  dodajemy  $(q, \varepsilon, A, q, B_1 \dots B_n)$ . W ten sposób nasz stos określa kolejne nieterminale, jakie mamy do dyspozycji.

Pokażemy, że  $A \rightarrow_G^* w$  wtedy i tylko wtedy, gdy  $(q, w, A) \vdash_P^* (q, \varepsilon, \varepsilon)$ . Jeśli  $A \rightarrow_G w$ , to teza zachodzi z konstrukcji. Dla dłuższego wywodu mamy  $A \rightarrow_G B_1 \dots B_n \rightarrow_G^* w_1 \dots w_n$  dla  $w = w_1 \dots w_n$ . Z założenia indukcyjnego mamy  $(q, w_i, B_i) \vdash_P^* (q, \varepsilon, \varepsilon)$ , a więc  $(q, w, B_1 \dots B_n) \vdash_P^* (q, \varepsilon, \varepsilon)$ . Wraz z  $(q, w, A) \vdash_P (q, w, B_1 \dots B_n)$  daje to tezę. W drugą stronę indukujemy się po ilości przejść automatu. Jeśli  $(q, w, A) \vdash_P (q, \varepsilon, \varepsilon)$ , to mamy  $A \rightarrow w$ . Dla większej ilości przejść jest  $(q, w, A) \vdash_P (q, w, B_1 \dots B_n) \vdash_P^* (q, \varepsilon, \varepsilon)$ . Istnieje podział  $w = w_1 \dots w_n$  taki, że  $(q, w_i, B_i) \vdash_P^* (q, \varepsilon, \varepsilon)$ . Zatem z założenia indukcyjnego  $B_1 \dots B_n \rightarrow_G^* w_1 \dots w_n$ , co kończy dowód.

( $\Leftarrow$ ) Niech  $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$  będzie PDA akceptującym  $L$  stosem pustym. Skonstruujemy odpowiednią gramatykę  $G$ . Będą do niej należeć stan startowy  $S$  oraz stany postaci  $N_{pXq}$  dla każdego  $p, q \in Q$  oraz  $X \in \Gamma$ . Do tego dla każdego stanu  $p \in Q$  mamy produkcję  $S \rightarrow N_{q_0 Z_0 p}$  oraz jeśli  $(q, a, X, r, Y_1 \dots Y_k) \in \delta$ , to dla każdej  $k$ -elementowej listy stanów  $r_1 \dots r_k$  tworzymy produkcję  $N_{qXr_k} \rightarrow aN_{r_1 Y_1 r_1} N_{r_1 Y_2 r_2} \dots N_{r_{k-1} Y_k r_k}$ . Jeśli  $(q, a, X, r, \varepsilon) \in \delta$ , to  $N_{qXr} \rightarrow a$ .

Pokażemy, że  $N_{qXp} \rightarrow_G^* w$  wtedy i tylko wtedy, gdy  $(q, w, X) \vdash_P^* (p, \varepsilon, \varepsilon)$ . To w szczególności da nam tezę. Jeśli  $(q, w, X) \vdash_P (p, \varepsilon, \varepsilon)$  (w jednym kroku), to mamy  $N_{qXp} \rightarrow_G w$ . Jeśli liczba kroków automatu jest większa niż jeden, to mamy  $(q, w, X) \vdash_P (r_0, v, Y_1 \dots Y_k) \vdash_P^* (p, \varepsilon, \varepsilon)$  dla  $w = av$  i istnieje podział  $v = v_1 \dots v_k$  i stany  $r_1, \dots, r_k$  takie, że  $r_k = p$ ,  $(r_{i-1}, v_i \dots v_k, Y_i \dots Y_k) \vdash_P^* (r_i, v_{i+1} \dots v_k, Y_{i+1} \dots Y_k)$ , czyli  $(r_{i-1}, v_i, Y_i) \vdash_P^* (r_i, \varepsilon, \varepsilon)$ . Zatem w  $G$  mamy wywód  $N_{qXp} \rightarrow_G aN_{r_0 Y_1 r_1} \dots N_{r_{k-1} Y_k r_k} \rightarrow_G^* av_1 \dots v_k$ .

Podobnie jeśli mamy  $N_{qXp} \rightarrow_G w$ , to mamy  $(q, w, X, p, \varepsilon) \in \delta$ . Dla dłuższego wywodu mamy  $N_{qXp} \rightarrow_G aN_{r_0 Y_1 r_1} \dots N_{r_{k-1} Y_k r_k} \rightarrow_G^* w$  dla  $r_k = p$ . Możemy zapisać  $w = av_1 \dots v_k$  dla  $N_{r_{i-1} Y_i r_i} \rightarrow_G^* v_i$ . Z założenia indukcyjnego  $(r_{i-1}, v_i, Y_i) \vdash_P^* (r_i, \varepsilon, \varepsilon)$ , a więc  $(r_0, v_1 \dots v_k, Y_1 \dots Y_k) \vdash_P^* (r_k, \varepsilon, \varepsilon)$ . Jednocześnie mamy  $(q, w, X) \vdash_P (r_0, v_1 \dots v_k, Y_1 \dots Y_k)$ , co kończy dowód.  $\square$

## 9. Języki bezkontekstowe

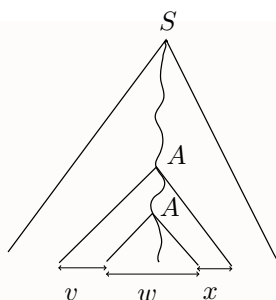
2025-03-23

**Propozycja 9.** Niech  $G$  będzie gramatyką bezkontekstową w postaci normalnej Chomsky'ego. Jeśli w drzewie wywodu słowa  $w$  każda ścieżka ma długość co najwyżej  $n$ , to zachodzi  $|w| \leq 2^{n-1}$ .

**Dowód.** Dowód indukcją po długości najdłuższej ścieżki w drzewie. Jeśli ma ona długość 1, to drzewo składa się z korzenia i jego jednego dziecka etykietowanego terminalem, mamy  $|w| = 1$ . Jeśli jest dłuższa, to korzeń ma dokładnie dwójkę dzieci (bo  $G$  jest w CNF). Oba poddrzewa ukorzenione w tych dzieciach mają długość najdłuższej ścieżki co najwyżej  $n-1$ , a więc z założenia indukcyjnego mamy  $|w| \leq 2^{n-2} + 2^{n-2} = 2^{n-1}$ .  $\square$

**Lemat 4 (O pompowaniu).** Niech  $L$  będzie językiem bezkontekstowym. Wtedy istnieje takie  $n \in \mathbb{N}$ , że dla każdego  $z \in L$  z  $|z| \geq n$  istnieje postać  $z = uvwxy$  taka, że  $|vwx| \leq n$ ,  $|vx| \geq 1$  i  $\forall i \in \mathbb{N} uv^iwx^iy$ .

**Dowód.** Załóżmy, że gramatyka  $G = (V, \Sigma, P, S)$  języka  $L$  jest w CNF (dla języków  $\emptyset, \{\varepsilon\}$  teza oczywiście zachodzi, a pozostałe języki mają taką gramatykę). Wybieramy  $n = 2^m$ , gdzie  $m = |V|$ . Wtedy w drzewie wywodu słowa  $z$  istnieje ścieżka długości co najmniej  $m+1$ , a więc pewien nieterminal występuje na niej co najmniej dwa razy. Niech  $A$  będzie pierwszym powtarzającym się nieterminalem (patrząc od dołu ścieżki).



Rysunek 3: Drzewo rozważanego słowa.

Niech  $w$  będzie częścią słowa generowaną przez dolne wystąpienie  $A$ . Niech  $vwx$  będzie częścią słowa generowaną przez górne wystąpienie  $A$ , a  $uvwxy$  całym słowem. Mamy  $|vwx| \leq n$ , bo inaczej  $A$  nie byłby pierwszym powtarzającym się nieterminalem (ścieżka byłaby za długa). Do tego  $vx \neq \varepsilon$  – górne wystąpienie  $A$  ma dwójkę dzieci, z których co najmniej jedno nie jest dolnym  $A$  i generuje jakieś litery. Zauważmy, że poddrzewo dolnego  $A$  można zastąpić poddrzewem górnego  $A$  i można to zrobić dowolną liczbę razy (lub zastąpić poddrzewo górnego  $A$  poddrzewem dolnego  $A$ ), a więc  $uv^iwx^iy \in L$  dla każdego  $i$ , bo ma odpowiednie drzewo wyvodu.  $\square$

### Twierdzenie 21. Język

$$L = \{a^n b^n c^n : n \in \mathbb{N}\}$$

nie jest bezkontekstowy.

**Dowód.** Niech  $n$  będzie stałą z lematu o pompowaniu. Rozważmy słowo  $a^n b^n c^n$  i założmy nie wprost, że ma ono postać  $uvwx$  jak w lemacie o pompowaniu. Jeśli  $vwx = a^p$ , to do języka będzie należeć słowo  $uvw = a^{n-|v|}b^n c^n$  – sprzeczność. Podobnie, gdy  $vwx$  składa się w całości z dwóch pozostałych liter. Jeśli  $vwx = a^p b^q$ , to do języka będzie należeć słowo  $a^{n-l}b^{n-k}c^n$  dla pewnych  $l, k$ . Podobnie dla kombinacji  $b^p c^q$ . Ograniczenie na wielkość  $vwx$  powoduje, że są to wszystkie możliwości.  $\square$